协同验证与性能分析

西安电子科技大学　香山芯锐

以 NEMU 和标准 RVV 基线建立正确性基础，先测量完整操作延迟，再分析含访存任务与同输出优化对照。

赛题 3.1.1、3.1.2、3.2、3.3 | 实验材料截至 2026-10-30 | 第五版 · 2026-11-04

7 NEMU 扩展与差分验证

本项目已在 NEMU 中加入 vdot.vv 的译码和执行语义，并完成参考库接口检查与 RTL/NEMU 整核协同验证。参考模型 API 通过 268 例，另有 10 例因配置不适用单列；修复版整核的 SMOKE、EXCEPTION、OVERLAP 和 PIPELINE 套件全部通过。

验证分三步配合进行。应用自检把输出与独立算出的答案比较；DiffTest 在同步点比较 RTL 与 NEMU 的寄存器和相关状态；波形则记录指令何时执行、发生异常、取消或退休。RTL 和 NEMU 分别按同一份指令规格实现，既核对点积答案，也核对异常与状态更新。

本章对应赛题 3.1.1 和 3.1.2，说明参考模型实现、构建与加载关系、验证结果，以及短 VL 异常中 mtval 失配的定位过程。

7.1 NEMU 与 RTL 共用 v0.1 指令规格

参考模型与 RTL 遵循同一份项目 v0.1 指令规格，合法配置范围和状态更新规则一致〔1〕。

在向量执行权限可用的前提下，合法配置要求 vill=0、SEW=8、LMUL=1、vstart=0，VL 为 0 或 8～16，vtype 保留高位为零。合法非零 VL 固定计算源低八项，精确和写入低 32 位、高 96 位清零；合法 VL=0 保持完整旧目的。类型、权限和重启条件的检查排在长度分支之前。

RTL 基于 XiangShan 16ae9ddc，叠加 VDot 和异常修复；参考模型基于 NEMU a48a11f2，叠加独立的语义实现。完整版本和二进制文件校验值见阶段三复现索引。

7.2 custom-0 译码规则与 def_EHelper(vdot)

NEMU 的改动包括译码入口、执行 ID 注册、头文件接入和新增语义函数。custom-0 的末七位为 `0001011`，funct6 和 funct3 为零，其余字段解析 vm 及寄存器编号。译码规则如下：

```
#ifdef CONFIG_RVV
  def_INSTR_IDTAB("000000 ? ????? ????? 000 ????? 00010 11", OP_V  , vdot);
  def_INSTR_IDTAB("??????? ????? ????? ??? ????? 10101 ??", OP_V  , OP_V);
#endif // CONFIG_RVV
```

第一条规则匹配 custom-0 编码，告诉 NEMU 执行 vdot；第二条保留原有的标准 OP-V 处理。新增执行 ID 注册在 isa-all-instr.h，vcompute.h 引入 vdot.h。这样，NEMU 识别机器码后，就能调用 `def_EHelper(vdot)` 执行对应规则。

执行函数如下，摘自 vdot.h 源码〔2〕：

```
def_EHelper(vdot) {
  require_vector(true); // host/guest VS and vill, before the vl=0 path
  if ((vtype->val >> 8) != 0 || vtype->vsew != 0 || vtype->vlmul != 0 ||
      vstart->val != 0 || (vl->val != 0 && (vl->val < 8 || vl->val > 16))) {
    longjmp_exception(EX_II);
  }
  const uint32_t word = s->isa.instr.val;
  const unsigned vd = (word >> 7) & 31;
  const unsigned vs1 = (word >> 15) & 31;
  const unsigned vs2 = (word >> 20) & 31;
  const unsigned vm = (word >> 25) & 1;
  if (vl->val != 0) {
    // Snapshot before any write, including vd=v0 and all source overlaps.
    const uint64_t a = cpu.vr[vs1]._64[0];
    const uint64_t b = cpu.vr[vs2]._64[0];
    const uint64_t mask = cpu.vr[0]._64[0];
    int32_t sum = 0;
    for (unsigned i = 0; i < 8; ++i) {
      const int ai = ((int)((a >> (8*i)) & 255) ^ 128) - 128;
      const int bi = ((int)((b >> (8*i)) & 255) ^ 128) - 128;
      if (vm || ((mask >> i) & 1)) sum += ai * bi;
    }
    cpu.vr[vd]._64[0] = (uint32_t)sum;
    cpu.vr[vd]._64[1] = 0;
  }
  vstart->val = 0;
  vp_set_dirty(); // includes legal vl=0 and guest VS when virtualized
}
```

实现中有三处与边界测试直接相关：

- 合法性检查位于 VL=0 之前，空操作也不能绕过异常条件。
- 两个源和掩码先存入局部变量，再改写目的；vd 与源或 v0 重叠时仍读取旧值。
- `(uint32_t)sum` 写入低 64 位容器时高 32 位为零，另一个 64 位字也置零，恰好形成低 32 位结果与高 96 位零。

合法 VL=0 跳过目的更新，但仍执行成功路径的状态处理；`vp_set_dirty()` 复用 NEMU 已有的向量状态机制。源文件及译码入口统一保存在 NEMU 修改快照中〔3〕。

7.3 仿真器、参考库与测试镜像的构建和加载

协同仿真要准备三个文件：`emu` 模拟香山硬件，`riscv64-nemu-interpreter-so` 提供 NEMU 参考模型，RISC-V 测试镜像保存待运行程序。启动时，emu 加载测试镜像，`--diff` 指向参考库，`-F` 指向启动 Flash。程序在硬件模型和参考模型中执行，DiffTest 在同步点比较两

侧的架构状态。



图 7-1　验证结构。应用对照独立期望值；DiffTest 在退休、异常等同步点比较 RTL 与 NEMU 导出的架构状态。

DiffTest 通过 regcpy 在两侧传递状态。本配置的状态交换区共 1400 字节，其中向量寄存器区从第 800 字节开始，占 512 字节，对应 32 个 128 位寄存器。参考库 API 测试先写入指定状态，执行一条指令，再读回比较；整核测试则让真实程序驱动 RTL 与 NEMU，检查完整执行过程〔4〕。

验收同时查看参考库路径、DiffTest 启用、应用 PASS、GOOD TRAP 和被测程序退出码；脚本及日志收集的退出状态单列记录。完整构建与运行命令入口见交付与复现记录〔5〕。

7.4 API 268 例与整核四类套件

API 测试先给 NEMU 设置寄存器和 CSR 初值，再让它执行一条 VDot，最后读回 32 个向量寄存器及相关 CSR，与预期比较。它检查参考模型自身的行为；随后运行整核套件，检查 RTL 是否得到同样的结果。表 7-1 列出 API 测试的数量。

表 7-1　参考模型 API 检查

类别	数量	主要检查
合法非零 VL	93	目的结果及状态更新
合法 VL=0	40	完整旧目的保持及状态策略
非法配置	135	异常与相关状态保持
通过合计	268	上述三类
配置不适用	10	本参考库构建无法通过软件动态关闭 misa.V，单列

API 层使用的本地参考库与服务器参考库身份分别保存。十个不适用用例不计入通过数，原因在于参考库的 RVV 构建配置。每例记录见原始 API 结果〔6〕。

整核阶段再检查译码、执行、异常和退休的联合行为。表 7-2 统一列出定向套件；PIPELINE 小计为三项流水套件之和。

表 7-2　修复版整核差分套件

套件	用例数	目的	结果
SMOKE	7	正负数、极值、掩码、VL=0/16、完整目的与状态	通过
EXCEPTION	64	62 个预期陷阱、2 个合法对照；cause、mepc、mtval	通过
OVERLAP	60	15 种源、目的、掩码相等关系	通过
PIPELINE-DEPS	48	12 个依赖测试程序	通过
PIPELINE-MIXED	32	8 个混发测试程序	通过
PIPELINE-CANCEL-CSR	40	32 个分支场景、8 个精确异常场景	通过
PIPELINE 小计	120	上述三项流水套件	通过

Hello 与 VADD 在修复后同轮通过。等价及性能应用另行运行，其内核调用数按各自实验统计。上述整核定向结果来自修复回归与流水回归原始记录〔7〕〔8〕。

7.5 执行期非法异常的 mtval 失配与修复

短 VL 用例曾发现一个异常信息错误：RTL 正确报告了非法指令，却没有把故障机器码写入 mtval。NEMU 的 mtval 有正确编码，因此 DiffTest 能报出差异。补上机器码传递通路后，原来的 64 个异常用例全部通过。下面按现象、原因、修改和复测说明。

7.5.1 现象：DUT 的 mtval 为零

早期 emu 执行短 VL 用例 short-1-vm0 时，短 VL 按规格触发非法指令异常，但 DiffTest 报告：

```
mtval different at pc = 0x0080000218, right=0x0020818b, wrong=0
```

参考模型的 mtval 为故障机器码 `0x0020818b`，DUT 的 mtval 为零。结合反汇编和提交轨迹可以区分两个 PC：`0x80000218` 是前一条已提交的 `sd t0,40(s0)`，实际发生异常的 VDot 位于 `0x8000021c`。异常同步时，差分行保留最近的比较位置，`mepc=0x8000021c` 则精确记录了故障指令地址。

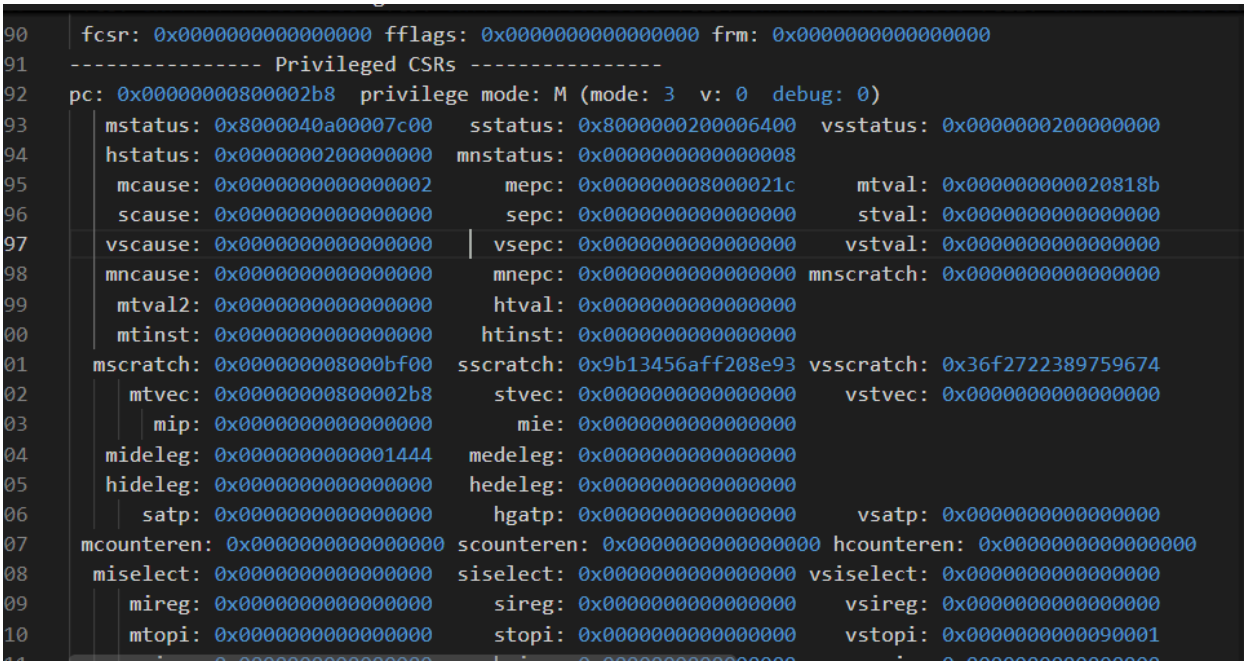


图 7-2　参考快照为 mcause=2、mepc=0x8000021c、mtval=0x0020818b。

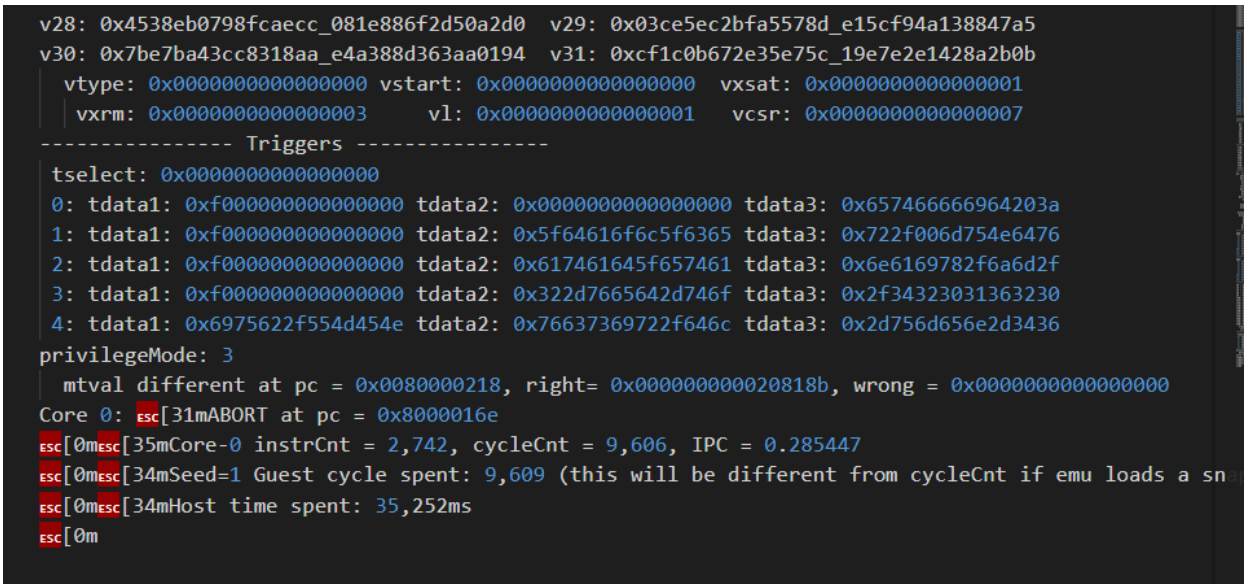


图 7-3　DUT 的 mtval 为零，参考值为故障机器码，差分检查随后中止运行。

7.5.2 原因：执行期非法异常拿不到指令字

VL=1 的非法条件是在执行期判断出来的。原有 trapInstInfo 主要传递译码期发现的非法指令，VDot 到达 ROB 队首时，异常类型已经送到，匹配的机器码却没有跟上，因此 mtval 读到零。需要修复的是故障指令信息通路。

7.5.3 改法：ROB 分配时保存可变编码字段和代际，队首重建指令字

修复后，ROB 在分配槽位时保存 VDot 的可变编码字段和代际。发生队首异常时，先确认槽位中的记录属于本次指令，再拼回机器码，交给 CSR 选择器。异常仲裁和 NEMU 语义保持原样，新增通路补上执行期异常所需的指令字。字段组成和容量见第 5 章 5.5。

7.5.4 回归：原异常用例 64 例 EXCEPTION 全部通过

修复生成新 emu 后，我们用原异常镜像与参考库配置回归：64 例 EXCEPTION 全部通过，随后 OVERLAP、SMOKE、Hello 与 VADD 也通过。原失败 emu 和日志保留用于追溯，成功结果对应修复后的仿真器版本。

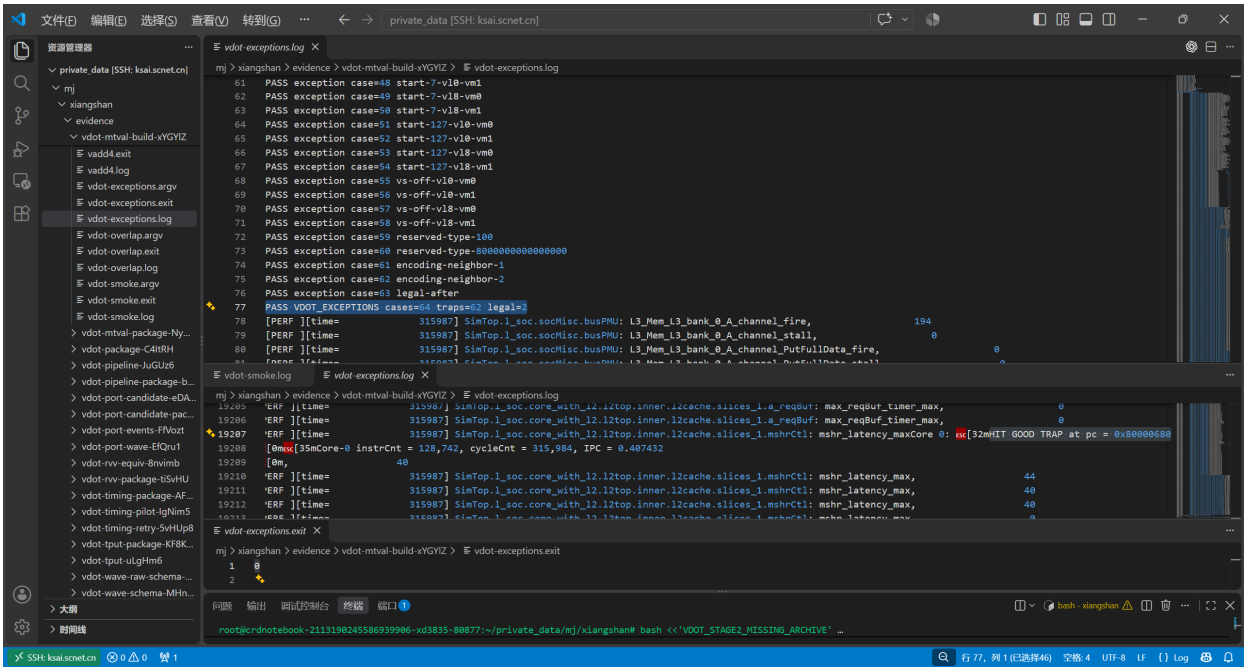


图 7-4　修复后异常套件正常结束，应用 PASS、GOOD TRAP 和退出状态共同确认本次回归结果。

该回归保留了失败快照、故障指令定位、RTL 修复和原用例复测记录。诊断与反汇编依据见差分诊断记录，具体修改见修复说明〔9〕〔10〕。

7.6 OVERLAP 与 PIPELINE 的运行记录

OVERLAP 专门让目的与源或掩码使用同一寄存器编号，检查计算是否仍读取执行前的旧值。PIPELINE-DEPS 检查后续指令使用前一条结果时的依赖；MIXED 检查与其他向量指令交错执行；CANCEL-CSR 检查分支取消和精确异常。下列日志与表 7-2 对应，内部时间关系见第 6 章波形。

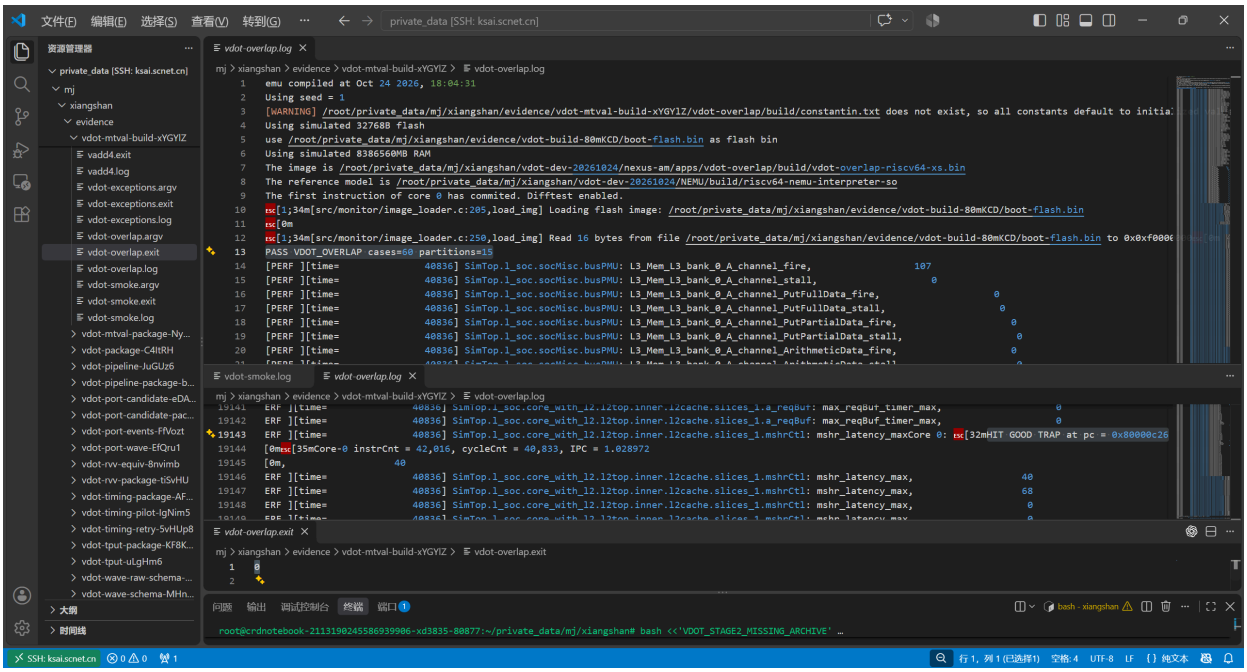


图 7-5　OVERLAP 套件通过，保留 DiffTest、GOOD TRAP 及退出记录。

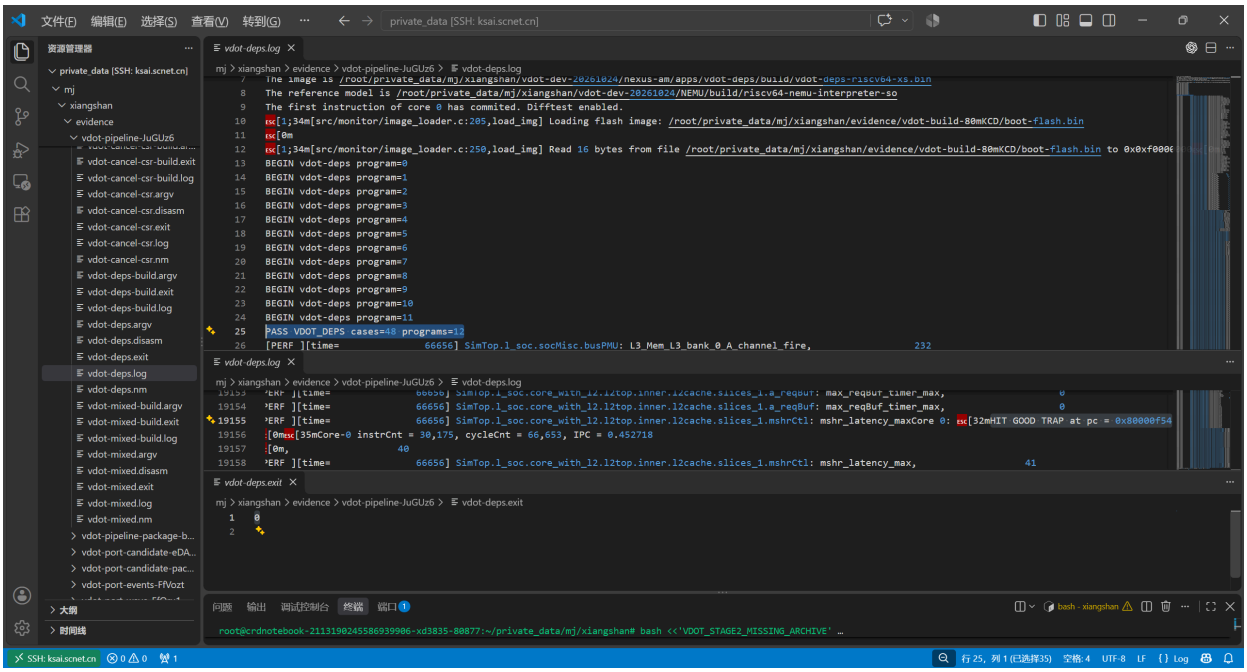


图 7-6　PIPELINE-DEPS 的依赖程序汇总与正常结束记录。

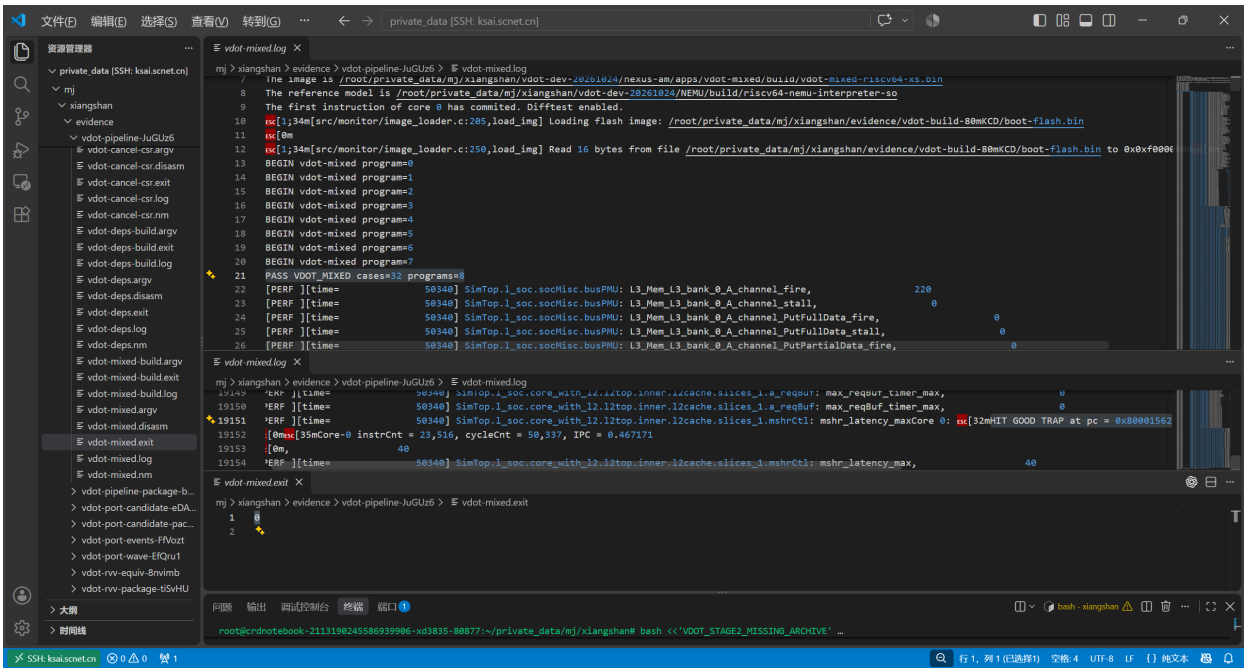


图 7-7　PIPELINE-MIXED 检查选定混发场景的架构结果。

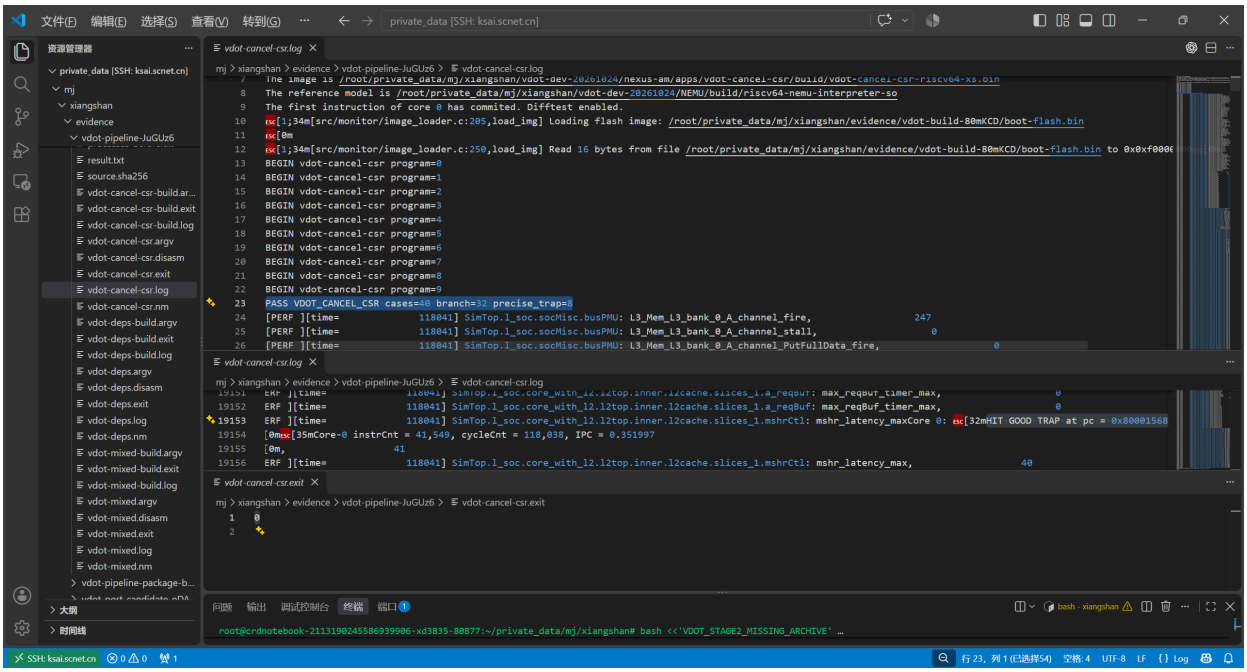


图 7-8　PIPELINE-CANCEL-CSR 分别记录分支和精确异常场景。

这些记录表明，定向套件中的 RTL 与参考模型架构状态一致。第 8 章据此构造标准 RVV 组合，并检查三条路径在统一调用约定下的完整结果和状态，为性能比较准备基线。

8 标准 RVV 等价基线

RVV-m1、RVV-mf2 两条标准向量指令组合已完成与 VDot 的等价检查：144 种输入配置、432 次内核调用的结果和指定状态一致，本地参考执行、完整 C 检查及服务器 RTL/NEMU 协同运行均通过。

公平比较周期前，先让三条实现完成同一件事：算出相同点积，得到相同的完整目的寄存器，VL=0 时保留旧目的，并在返回时恢复约定的配置。本章说明怎样用标准 RVV 指令完成这些操作，以及怎样核对三条实现的结果。

8.1 kernel_end 比较点与寄存器约定

三条实现都执行到汇编标签 kernel_end 再比较状态。此时核心计算已经结束，测试程序还没有为存储结果而重新配置向量。入口统一为合法 M 态、e8/m1、vstart=0，向量状态可用。

调用约定中，v0 放掩码，v1/v2 放两个源，v3 接收结果。v8/v9/v16 和 t0 留给实现临时使用，调用者不要求它们保持原值。测试在 kernel_end 检查约定需要保持的寄存器和状态，随后才存储结果。

每条实现先与独立算出的期望值比较，再与另外两条实现比较。这样能发现某条实现单独出错，也能检查三条实现是否一起算成了同一个错误答案。这套调用约定用于性能对比。源与目的寄存器任意重叠、非法配置等情况，由第 7 章的定向套件另行检查。

8.2 vwmul.vv 加 vwredsum.vs：两次加宽得到 INT32

标准 RVV 分两步计算：vwmul.vv 把两组 INT8 相乘，得到 INT16 乘积；vwredsum.vs 再把八个 INT16 乘积相加，得到 INT32 和。归约种子，也就是求和的起始值，设为零。计算前先把 v3 全部清零；归约时采用尾部保持策略，让低 32 位接收总和，其余位仍为零。

乘法和归约的元素宽度不同，需要切换 SEW/LMUL。在 VLEN=128 时，八个 INT8 只占半个向量寄存器。RVV-m1 按 m1 配置乘法，加宽后的目的使用 v8/v9 这一组；RVV-mf2 从 mf2 开始，加宽后恰好使用一个 m1 的 v8。两条基线计算相同的八项点积，保留它们是为了比较不同寄存器分组的执行成本。

表 8-1　两条基线的乘法与归约配置

基线	乘法配置	乘积存放	归约配置
RVV-m1	e8/m1，VL=8	EMUL=2，v8/v9	e16/m2，VL=8
RVV-mf2	e8/mf2，VL=8	EMUL=1，v8	e16/m1，VL=8

乘法生成完整的有符号乘积，掩码在归约时选择有效项。VL=0 跳过计算并保留旧目的，末尾恢复调用者的 vl/vtype。标准指令行为依据 RVV 1.0 规格，实际实现以本项目汇编为准〔11〕。

8.3 三条核心段：1 条对 9 条静态指令

在本章调用约定下，VDot 核心段为一条自定义指令，两条 RVV 基线的非零路径各为九条静态指令。下文列出具体代码；周期成本在第 9 章测量。

共同测试壳已装入源、掩码和入口配置，a3/a4 分别保存原 VL/vtype。下面列出 vm=0 的三个核心段。

VDot 由单条自定义指令完成：

```
    .word 0x0020818b
```

该字对应 `vdot.vv v3,v2,v1,v0.t`，操作数顺序为 vd、vs2、vs1。

RVV-m1 显式完成分支、配置、乘法、清零、归约和恢复：

```
    beqz a3,rvv_m1_vm0_restore
    vsetivli zero,8,e8,m1,tu,mu
    vwmul.vv v8,v1,v2
    vsetivli zero,4,e32,m1,tu,mu
    vmv.v.i v3,0
    vmv.v.i v16,0
    vsetivli zero,8,e16,m2,tu,mu
    vwredsum.vs v3,v8,v16,v0.t
rvv_m1_vm0_restore:
    vsetvl zero,a3,a4
```

RVV-mf2 改变乘法和归约的分组，其余职责相同：

```
    beqz a3,rvv_mf2_vm0_restore
    vsetivli zero,8,e8,mf2,tu,mu
    vwmul.vv v8,v1,v2
    vsetivli zero,4,e32,m1,tu,mu
    vmv.v.i v3,0
    vmv.v.i v16,0
    vsetivli zero,8,e16,m1,tu,mu
    vwredsum.vs v3,v8,v16,v0.t
rvv_mf2_vm0_restore:
    vsetvl zero,a3,a4
```

在 e32/m1、VL=4 下，`vmv.v.i` 将目的 v3 和归约种子 v16 完整清零。切换配置后，`vwredsum.vs` 把和写入 v3 的第一个 32 位元素，tu 让剩余位保留先前的零。最后 `vsetvl` 恢复入口 VL/vtype。无掩码 vm=1 的版本只需在归约指令中省去 `v0.t`。



图 8-1　VDot 与 RVV 非零路径的静态指令组织。五个 RVV 步骤合计 9 条指令；m1 与 mf2 的区别在乘法和归约配置。

九条指令分别完成长度判断、三次向量配置、一次乘法、两次清零、一次归约和一次配置恢复。VDot 将这些约定的计算与结果处理合在一条指令中。指令数为九比一；实际少用多少周期，还要看依赖等待、配置切换和调度，第 9 章按完整操作测量。

8.4 等价验证用例：6 组数据、144 种配置、432 次调用

测试先选六组数据，再分别组合两种 vm、三种 VL 和四种 vta/vma，得到 144 种输入配置。每种配置都运行 VDot、RVV-m1、RVV-mf2，共调用 432 次。检查器核对每条实现的完整结果、源寄存器和指定状态。

六组数据覆盖正数、混合符号、正负极值、零掩码和稀疏掩码。旧目的与源高半部填入非零值，用来识别错误清零、错误保持和高半输入污染。

表 8-2　测试数据与配置组合

维度	取值
数据集	6 组
vm	0、1
VL	0、8、16
vta/vma	4 种组合
输入配置数	6×2×3×4=144
每配置实现路径	VDot、RVV-m1、RVV-mf2
内核调用总数	144×3=432



图 8-2　测试数据与配置的组合关系。144 种输入配置各执行三条实现，共 432 次内核调用。

Python 整数模型先算出独立期望。C 检查器逐字节比较 v0～v3 的全部 64 字节：v0 和两源应保持，v3 应满足结果布局或 VL=0 保持规则。随后检查 VL/vtype 是否恢复，VS 是否为 Dirty，vstart 是否为 0，vcsr/fflags 是否保持。

六个内核的反汇编也逐一检查，确认两条标准路径实际执行加宽乘法和归约，没有混入 custom-0。还故意改坏参考模型 RAM 中的一个期望字节，C 检查器随即返回 1，确认比较失败时确实会报错。

8.5 整核等价验证结果

服务器构建、运行、脚本及 tee 退出码均为 0，GOOD TRAP 位于 `0x80000858`；本地参考执行和完整 C 检查也全部通过。

应用独立构建后交给修复版 emu，显式加载源码 NEMU 和共同 Flash。服务器运行使用 seed=1，最大 300 万仿真周期、1800 秒墙钟上限；实际构建 3.726 秒，整核运行 680.930 秒。这些主机时间只用于安排复现，处理器周期另由计时内核读取。

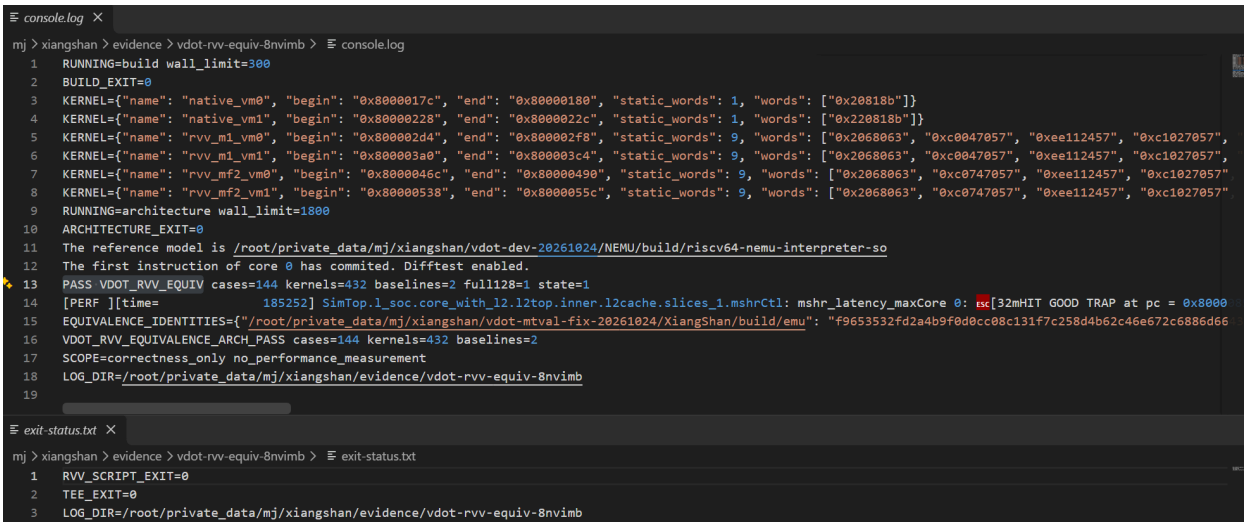


图 8-3　两条 RVV 基线与 VDot 的等价验证通过。正常结束地址在完整日志中为 0x80000858，位于本截图右侧裁剪范围之外。

三条实现都通过相同的结果和状态检查后，再比较周期。第 9 章会把 RVV 为满足约定而执行的配置、清零、归约和恢复一起计时，依次测量前后有依赖的连续点积、可以重叠执行的批处理，以及包含加载和存储的任务。

本地结果、服务器核验、应用源码与实际构建运行入口均保留在配套工程中〔12〕〔13〕〔14〕〔15〕。

9 延迟测量与软件优化分析

数据已经装入寄存器时，若后一次点积必须读取前一次结果，每增加一次完整操作，VDot 增加 7 周期，RVV-m1 增加 55 周期，RVV-mf2 增加 47 周期。加入加载和存储后，收益随程序安排变化：初版逐点输出基本持平；优化输入加载和 2×2 输入复用后，在相同输出要求下，RVV-mf2 的总周期分别是 VDot 的 1.24 倍和 1.32 倍。

本章先测依赖链：每次把结果交给下一次点积，查看串行执行的完整操作延迟。再固定输入、轮流使用八个目的寄存器，观察多条操作重叠执行时的吞吐。最后加入内存加载、结果存储和共享输入，比较软件安排怎样影响整个任务的用时。每组都在相同输入输出要求下比较 VDot 与标准 RVV。

9.1 依赖链延迟：每多做一次点积增加几周期

9.1.1 增加链长，分离固定计时开销

读取计数器、设置计时边界和取用最终结果，本身就要花时间。预检中，什么点积也不做的空窗口为 74 周期，只做一次 VDot 的窗口为 81 周期。为减少这些固定步骤的影响，主实验分别连续做 1、8、32 次点积，用“多花的周期数”除以“多做的次数”，计算每增加一次操作的成本。

依赖链使用 v1 作为输入之一，同时把结果写回 v1。第一条 VDot 写出的新 v1，要被第二条读取；第二条写出的结果又被第三条读取。因此，后一次必须等前一次的结果，这就是前后操作的数据依赖。程序分别展开 N=1、8、32 次，输入加载、最后存储和打印放在计时窗口外；RVV 每次需要的配置、清零和恢复仍计入窗口。不同链长的周期差用于计算完整操作延迟。

9.1.2 周期、延迟、吞吐和加速比

处理器的 `mcycle` 记录仿真中的周期数，`minstret` 记录退休指令数；主机执行模拟器用了多少秒，另作运行时间记录。设 N 次点积的窗口周期为 C(N)，退休数为 I(N)，正式样本取中位数。表 9-1 给出计算方式。

表 9-1　本章的指标定义

指标	定义	含义
窗口周期 C(N)	结束 mcycle − 开始 mcycle，单位为周期	完整计时窗口的执行周期
退休数 I(N)	两次 minstret 读数之差，单位为条	计时窗口内退休的架构指令条数
增量成本 d	$[C(N_2)-C(N_1)]/(N_2-N_1)$，周期/次	每增加一次操作带来的周期增量
完整操作延迟 L	真实依赖链中测得的 d	串行依赖下每次完整操作的周期成本
延迟加速比	L_RVV/L_VDot	RVV 与 VDot 的完整操作延迟之比
增量吞吐比	独立批处理中 d_RVV/d_VDot	操作可重叠时，RVV 与 VDot 的平均周期成本之比
整窗比	同任务、同 N 的 C_RVV/C_VDot	包含公共开销的同任务整窗周期比

例如延迟比 47/7=6.71，表示在本依赖链中，RVV-mf2 每增加一次完整操作所需的周期是 VDot 的 6.71 倍。比较 VDot 自身优化前后时，则计算“原周期/新周期”或减少率 1−C_新/C_原。这两种比较的对象不同，后文分别列出。正式样本同时保留中位数和最小～最大值，便于看到运行波动。

9.1.3 预热与取样规则

正式计时前，先运行相同内核，让程序代码和数据被访问一次或多次，这一步称为预热。预热输出单独保留；后续正式样本才用于计算中位数和范围，减少首次访问对比较的影响。

同组 VDot 与 RVV 使用相同输入准备、预热、结果检查和采样规则，正式轮次轮换执行顺序。早期实验每变体预热一次，加载与复用补测增加为五次；每点正式样本数量见表 9-2。

表 9-2　各实验的样本安排，预热与正式总数包含空对照

实验	变体数	每变体预热	每点正式样本	正式样本总数
计时预检	6	1	6	36
依赖链	10	1	3	30
八路批处理	10	1	3	30
逐点输出小数组	10	1	3	30
分段点积两种组织	13	1	3	39
输入加载策略与轮转	7	5	9	63
2×2 输入复用	5	5	9	45

所有测量采用锁定的单核 MinimalConfig、VLEN=128、seed=1，复用同一修复版 emu、源码 NEMU 和 Flash。早期完整样本见归档逐轮表〔16〕，新增两组见补测结果〔17〕。

9.2 CSRRW 写 mscratch 作计时边界

在本次锁定的香山版本中，部分只读 CSR 指令可以与前后指令乱序执行。计时程序用写 mscratch 的 CSRRW 建立边界，限制计数读取与待测代码的相对顺序。待测代码后还安排一条使用最终结果的指令，让结束计时之前必须等到结果可用。次序如下：

```
外侧边界 → C0 → 边界 → I0 → 边界 → 负载 → 结果消费者
         → 边界 → I1 → 边界 → C1 → 外侧边界 → 保存与恢复
```



图 9-1　计时指令顺序与计数窗口。C0/C1、I0/I1 分别给出周期和退休指令数读数；B 为写 mscratch 的 CSRRW 边界。

C0、C1 是前后两次 mcycle 读数，I0、I1 是前后两次 minstret 读数。图中的“结果消费者”指使用最终计算结果的指令：预检用 `vmv.x.s` 把 v3 的结果取到整数寄存器，再检查低字节。mscratch 的入口值先放入 t6，返回前恢复并核对。周期差仍包含四个内部写边界、两次退休计数读取和结果消费者，因此保留空窗口作为共同开销的参照。

该边界针对本版本微架构的调度行为建立，实际汇编见计时程序〔18〕。含存储的实验还在窗口末尾执行一次 `fence rw,rw` 和结果读回，把最终输出完成的成本纳入比较。

预检包含空操作、整数指令序列及三条点积路径。每点六次正式样本周期一致，退休增量均为负载指令数加 4，结果见表 9-3。

表 9-3　计时预检结果

负载	负载指令数	窗口周期	退休增量
empty	0	74	4
addi16	16	86	20
addi64	64	134	68
VDot	1	81	5
RVV-m1	9	130	13
RVV-mf2	9	122	13

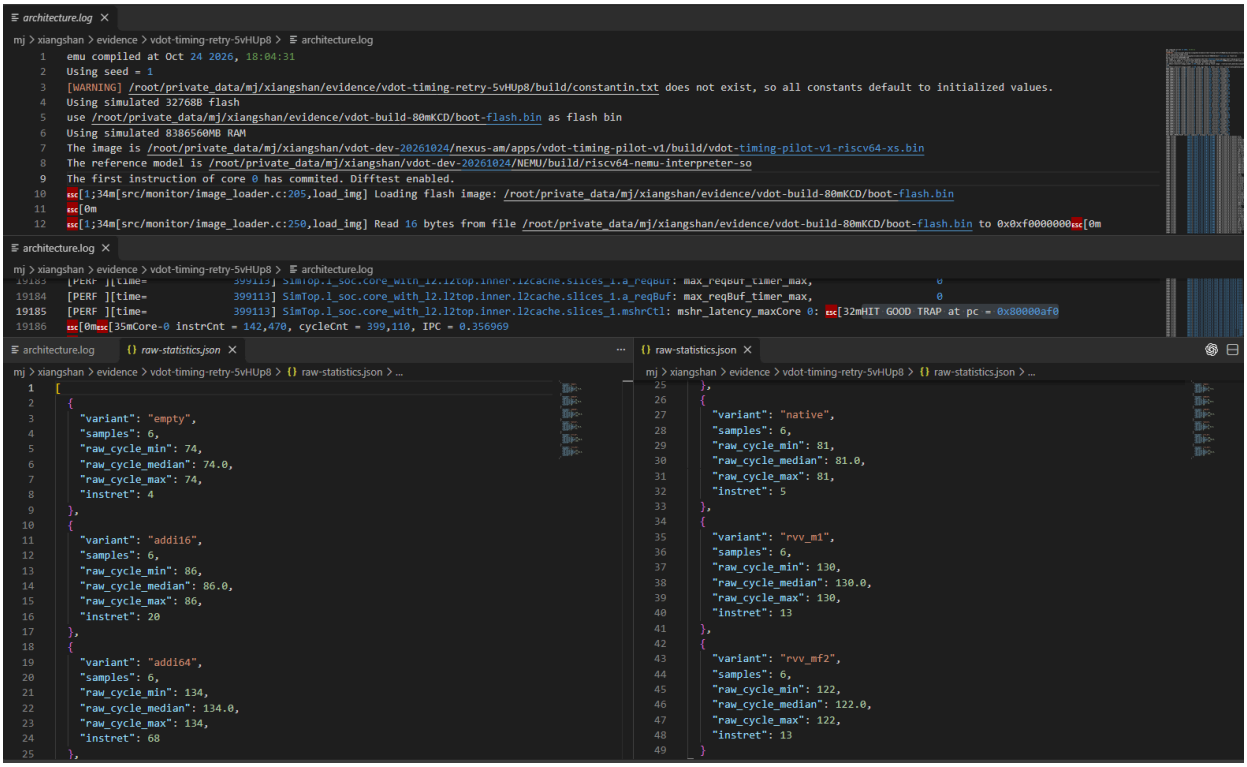


图 9-2　六组预检的周期与退休数，连同参考库、DiffTest 和正常结束信息。应用通过及退出状态保存在预检记录〔19〕。

9.3 依赖链延迟：7 对 55 / 47 周期

三条路径每次都读取 v1 并把结果写回 v1，后一次操作必须等待前一次的新值。VDot 用一条自定义指令；RVV 每次还要配置、乘法、清零、归约和恢复。表 9-4 列出做 1、8、32 次时的读数。每个点的三个正式样本一致，1→8 与 8→32 两段算出的每次新增周期也一致。

表 9-4　依赖链周期和退休数，单元格为“周期 / 条”

路径	N=1	N=8	N=32	1→8 增量	8→32 增量
VDot	81 / 5	130 / 12	298 / 36	7 周期/次	7 周期/次
RVV-m1	130 / 13	515 / 76	1835 / 292	55 周期/次	55 周期/次
RVV-mf2	122 / 13	451 / 76	1579 / 292	47 周期/次	47 周期/次

7 周期是后一次操作等到前一次结果、经调度并继续执行时，每多做一次完整点积增加的时间。第 6 章的 2 周期只从 VDot 功能单元接受输入算到输出完成。两项测量的起点和终点不同：前者包含后端依赖与调度，后者只看功能单元内部流水。

以 8 次增加到 32 次为例，多做了 24 次。VDot 从 130 增加到 298 周期，多用 168 周期，平均每次增加 168/24=7 周期。RVV-mf2 从 451 增加到 1579 周期，平均每次增加 (1579−451)/24=47 周期。因此两条 RVV 路径与 VDot 的延迟比分别为：

```
相对 RVV-m1：55 / 7 = 7.86 倍
相对 RVV-mf2：47 / 7 = 6.71 倍
```

三个测量点分别满足 C_VDot(N)=74+7N、C_m1(N)=75+55N、C_mf2(N)=75+47N。公式中的 74 或 75 是固定部分，N 前面的系数是每增加一次操作的周期。用长链减短链，再除以新增次数，可以消去各自的固定部分，无需假定三条实现的固定开销完全相同。

若直接比较 32 次操作的整个窗口，RVV-mf2/VDot 为 1579/298=5.30。这个比值把双方的固定计时开销也算进去，所以小于按新增操作计算的 6.71。后文称它为整窗比，即同一任务整个计时窗口的周期比。

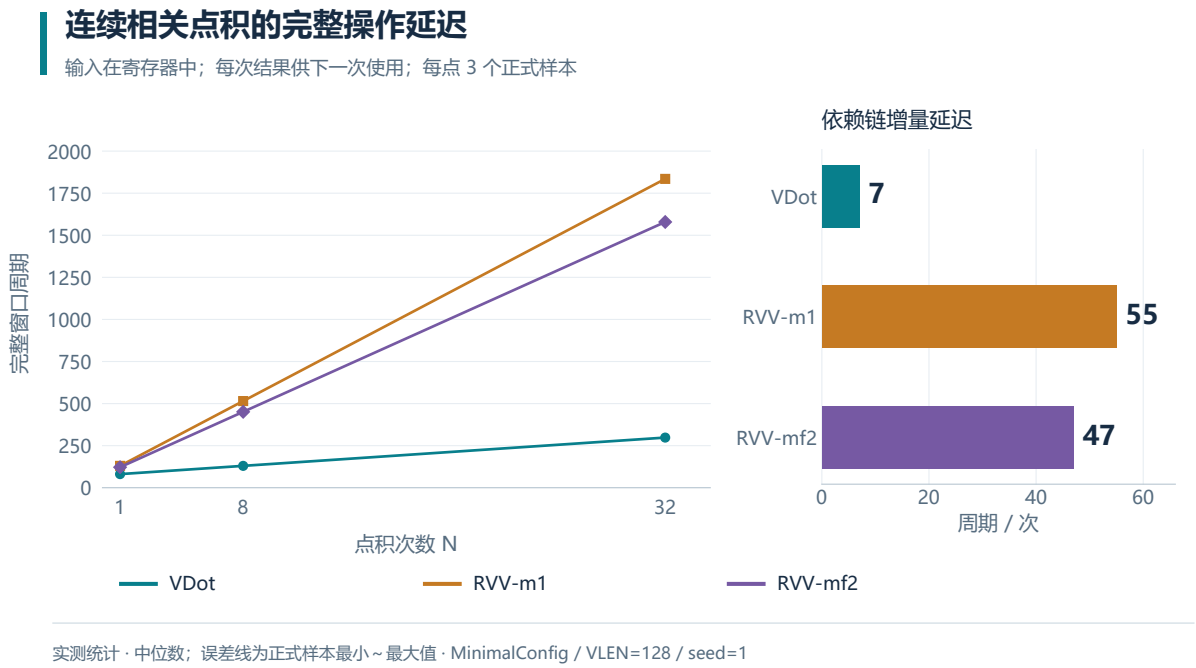


图 9-3　依赖链数据与斜率。VDot 两段均为 7 周期/次，两条 RVV 基线分别为 55、47 周期/次。

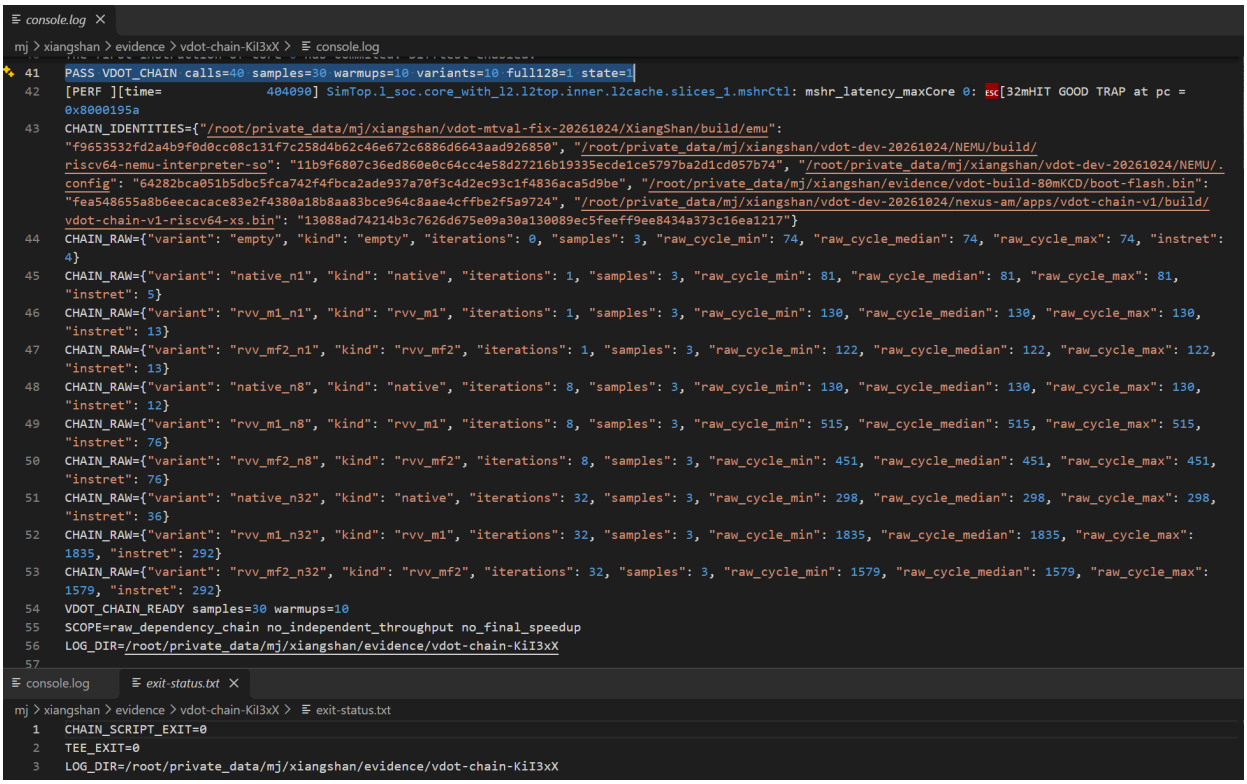


图 9-4　依赖链原始记录，包含周期、退休数、PASS 和退出状态。数据来源为原始统计〔20〕。

因此，在本配置和调用约定下，前后操作有数据依赖时，VDot 完成一次新增点积所需周期少于两条 RVV 组合。该结果用于 Latency 对比。配套材料保留源程序、全部样本和运行记录〔21〕〔22〕。

9.4 八路批处理：VDot 增量 1 周期/次

批处理实验固定以 v1、v2 为源寄存器，结果轮流写入 v24～v31。后续指令继续读取 v1、v2，不读取前一条的计算结果，因此避开了依赖链中相邻计算之间的结果数据依赖，可以观察连续执行时的吞吐。RVV 每八次操作共用乘法与归约配置。计时包含开始清零、结束恢复和八个结果的读取汇总；再次使用同一目的寄存器时，仍保留实现中声明的 oldVd 依赖。

表 9-5　八路批处理周期和退休数，单元格为“周期 / 条”

路径	N=8	N=32	N=128	8→32 增量	32→128 增量
VDot	96 / 26	120 / 50	216 / 146	1.00	1.00
RVV-m1	186 / 47	330 / 101	906 / 317	6.00	6.00
RVV-mf2	159 / 47	232 / 101	519 / 317	3.04	2.99

表中增量单位为周期/次，每个测量点的三个正式样本一致。VDot 从 32 次增加到 128 次，多做 96 次、也多用 96 周期，即每增加一次约增加 1 周期。RVV-m1 每次增加 6 周期。RVV-mf2 从 8 次到 128 次的平均值为 (519−159)/120=3；分成两段计算为 3.04 和 2.99，因为中间点比两端连线高 1 周期。

把 128 次操作的总周期直接相比，RVV-m1/VDot 为 906/216=4.19，RVV-mf2/VDot 为 519/216=2.40。该实验让多条操作有机会在流水中重叠，测的是连续处理的平均成本。VDot 平均每次增加 1 周期，与单次功能单元计算用 2 周期可以同时成立：前一条还在后级求和时，后一条可以进入前级。

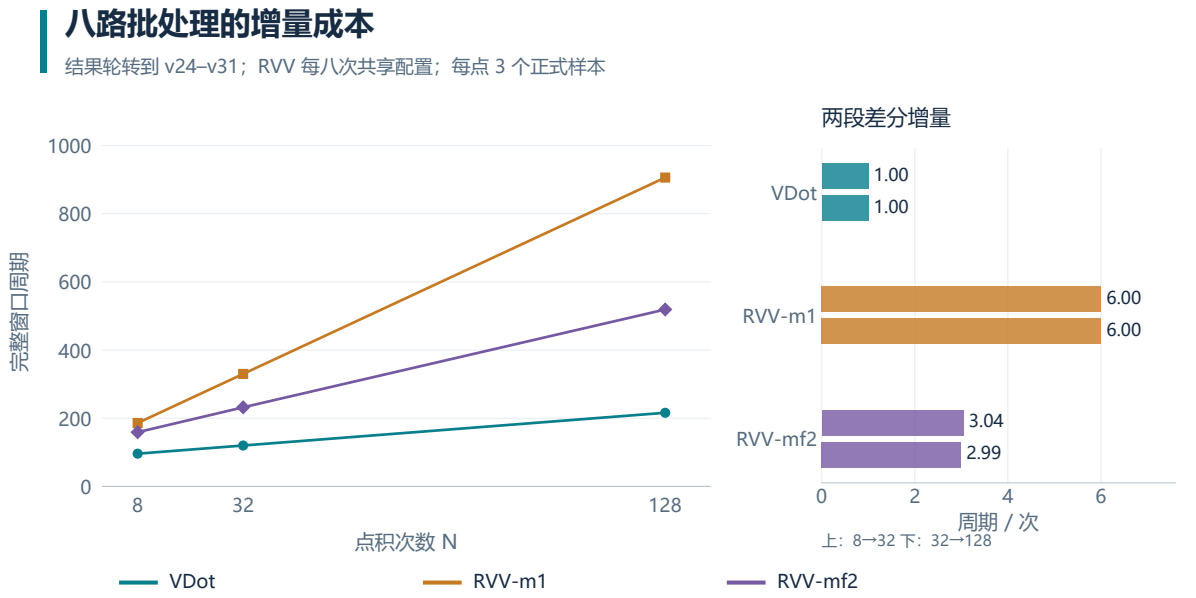


图 9-5　VDot 的增量为 1 周期/次；RVV-mf2 保留两段实际差分值。

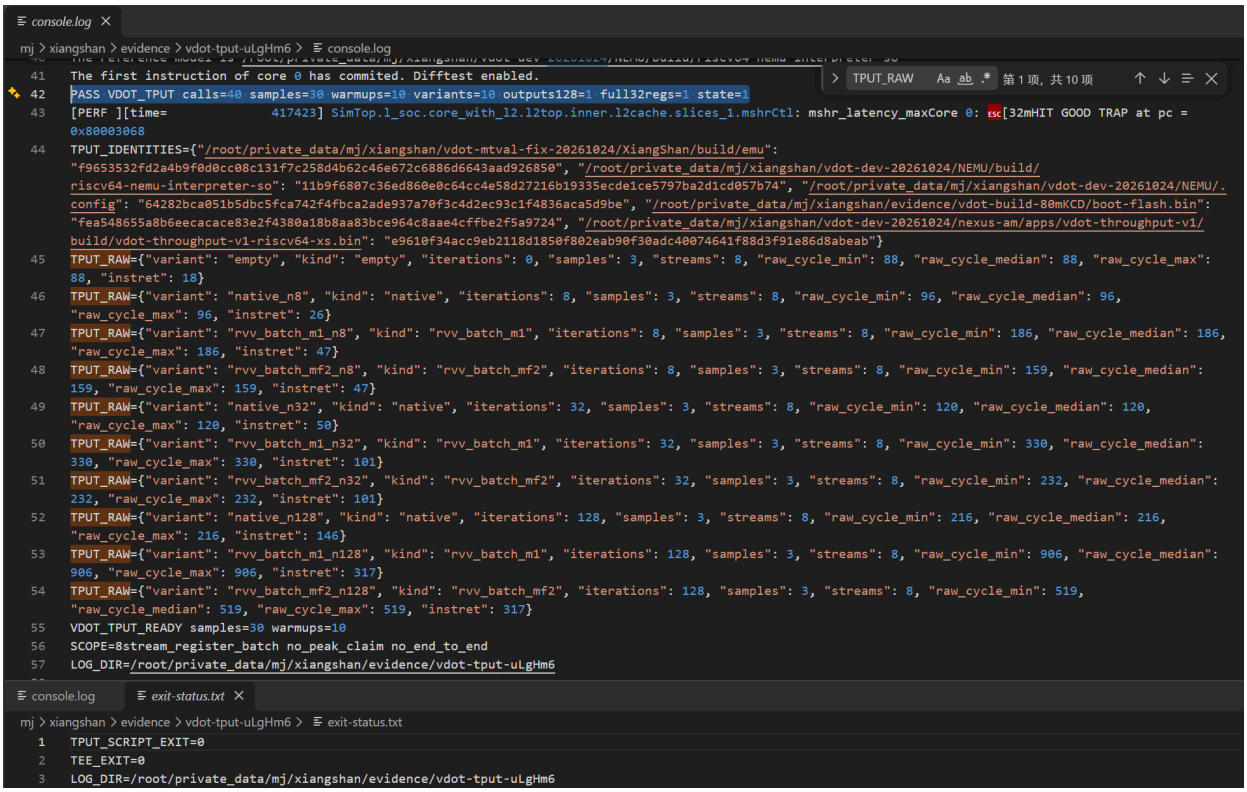


图 9-6　八路批处理实测结果，对应原始统计〔23〕。

寄存器实验分别给出了依赖链与批处理成本。下一组将输入加载和结果存储纳入窗口，检查完整数据流对执行时间的影响。

9.5 逐点输出小数组：N=128 时基本持平

小数组每次从内存读取两组各八字节输入，做一次点积，并保留完整 16 字节结果。每完成八次点积，集中存回八个结果；窗口末尾执行 fence 并读回末项。计时包括地址计算、加载、配置、计算、存储以及等待最终输出完成。初始化、预填输出、自检和打印放在窗口外。

表 9-6　逐点输出小数组的实测成本。周期为中位数，括号为出现波动时的最小～最大值；每点三个正式样本

路径	N	周期	退休数
VDot	8	287	65
VDot	32	674（674～676）	239
VDot	128	2239（2239～2240）	935
RVV-m1	8	320	84
RVV-m1	32	705	285
RVV-m1	128	2249	1089
RVV-mf2	8	312	84
RVV-mf2	32	698（696～698）	285
RVV-mf2	128	2241	1089

做 128 次时，VDot 用 2239 周期，RVV-mf2 用 2241 周期，只差 2 周期，基本持平。VDot 虽然少退休 154 条指令，但加载和存储等环节也占时间，减少算术指令没有按同样比例缩短总周期。整窗比为 2241/2239=1.0009；VDot 的整个窗口平均约 17.49 周期/次。

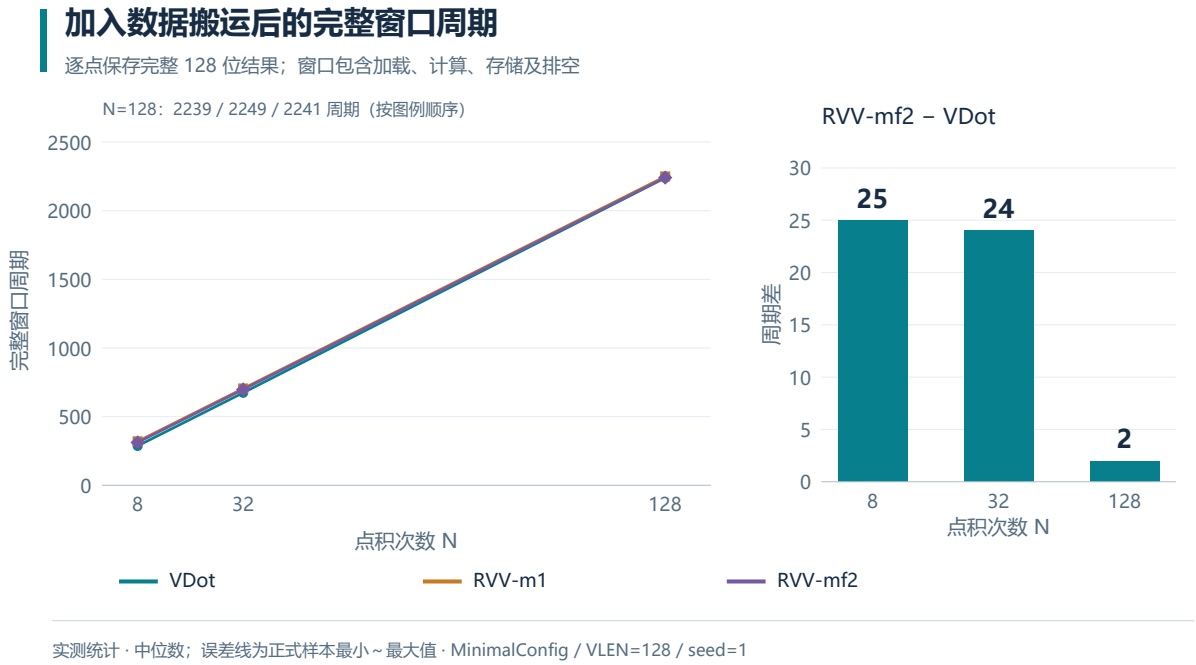


图 9-7　随着 N 增长，VDot 与 RVV-mf2 的周期差从 25、24 缩小到 2。

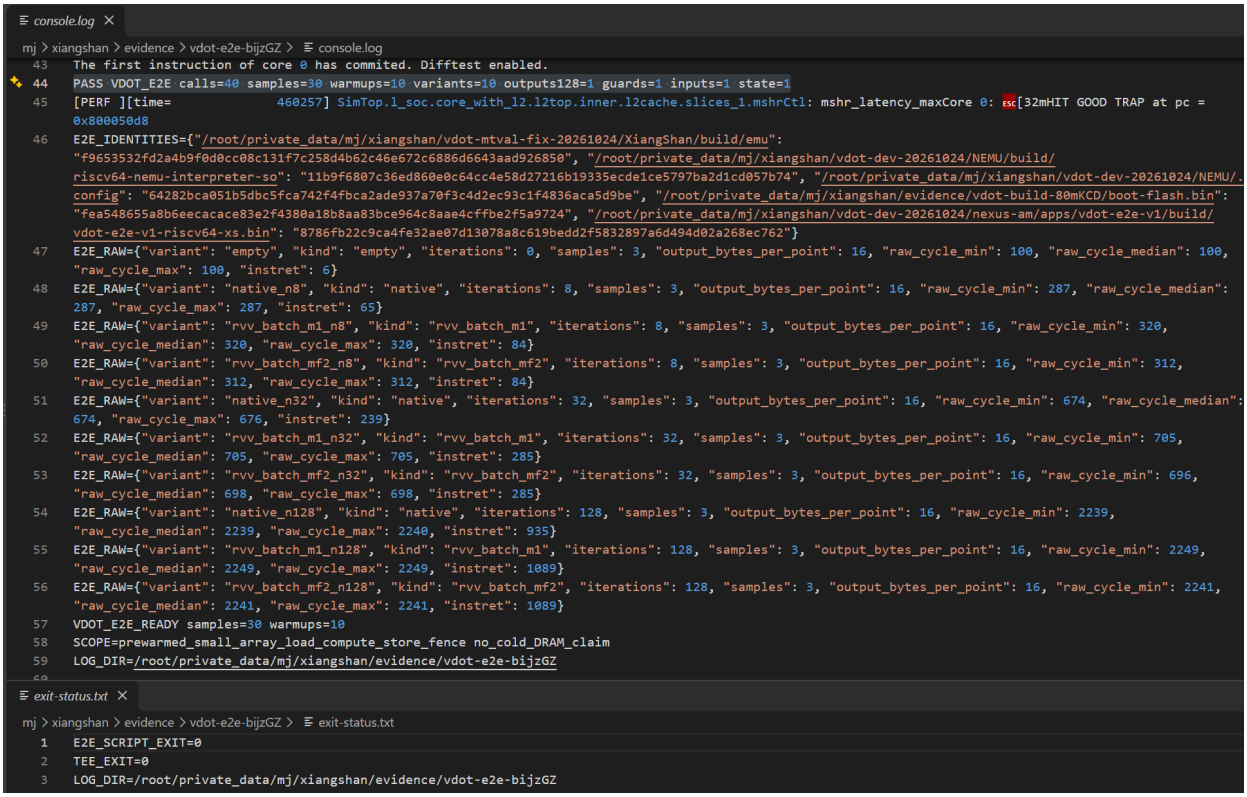


图 9-8　小数组原始统计及检查记录；完整数据见原始统计〔24〕。

为分析时间主要消耗在哪些环节，进一步查看应用全程计数器。日志中加载和存储等待活动较多，Sbuffer 请求有效次数与接受次数也有明显差异。

表 9-7　小数组应用全程计数，日志 time=460257

计数器	值	观察含义
ROB clock_cycle	460258	整个应用的计数范围
ROB commitInstr	210343	整个应用退休量
ROB waitLoadCycle	182322	存在较多加载等待活动
ROB waitStoreCycle	161379	存在较多存储等待活动
ROB waitFenCycle	860	包含 fence 等待
Sbuffer req_valid / req_fire	31229 / 16772	请求与接受次数不同
DCache req_valid / req_fire	3910 / 3903	该接口请求和接受计数

这些计数来自整个应用，包含预热、初始化、自检、打印和所有变体；不同等待项还可能同时出现。它们提示后续值得检查数据搬运，但不能直接用来分摊某个内核每一种停顿的周期。下一组实验因此保持输出要求相同，分别改变加载、存储和等待安排，直接比较总周期〔25〕。



图 9-9　逐点、分段与 2×2 任务的输出形式。图中分段示例取 G=16；每种软件优化在对应任务内部保持输出相同。

后续实验分别固定各自的输出要求，再改变程序安排：分段点积比较中间结果是否需要存入内存；加载实验比较怎样准备下一组输入；2×2 小块比较四次点积能否共用已加载的数据。每组先看同一实现优化前后的变化，再看 VDot 与同等优化 RVV 的差异。

9.6 分段点积：中间存储与寄存器归约

分段任务每次读取两组各 1024 字节输入，完成 128 次八项点积。每连续 G 次点积再相加，输出一个段和；G 取 8 或 16。比较两种做法：第一种算完点积就留在寄存器里累加，第二种先把每次完整点积结果存到 scratch 临时区，再读回来相加。第二种额外写 2048 字节、读 2048 字节，最终段和与第一种相同。

G=8 最终输出 16 个完整向量结果，G=16 输出八个。两种组织、三条实现各自比较；计时均包含加载、地址与循环控制、计算、归约、最终存储及末尾 fence/读回。

表 9-8　分段任务的相同输出对照。括号为正式周期范围，退休数按“寄存器 / 落存”列出

G	路径	寄存器归约周期	中间落存周期	退休数
8	VDot	2282（2282～2383）	3031（3030～3169）	859 / 1388
8	RVV-m1	2275（2273～2427）	3394（3394～3440）	1013 / 1542
8	RVV-mf2	2321（2320～2454）	3312（3312～3313）	1013 / 1542
16	VDot	2240（2240～2296）	3026（3026～3079）	843 / 1372
16	RVV-m1	2360（2359～2438）	3443（3443～3444）	997 / 1526
16	RVV-mf2	2333（2333～2405）	3263（3262～3312）	997 / 1526

采用寄存器归约后，VDot 在 G=8、G=16 下分别减少 24.7%、26.0% 的周期。两条 RVV 路径也因减少中间搬运而改善。在相同寄存器归约组织内，VDot 相对 RVV-mf2 的整窗比分别为 1.017、1.042，明显小于两种数据组织之间的差异。

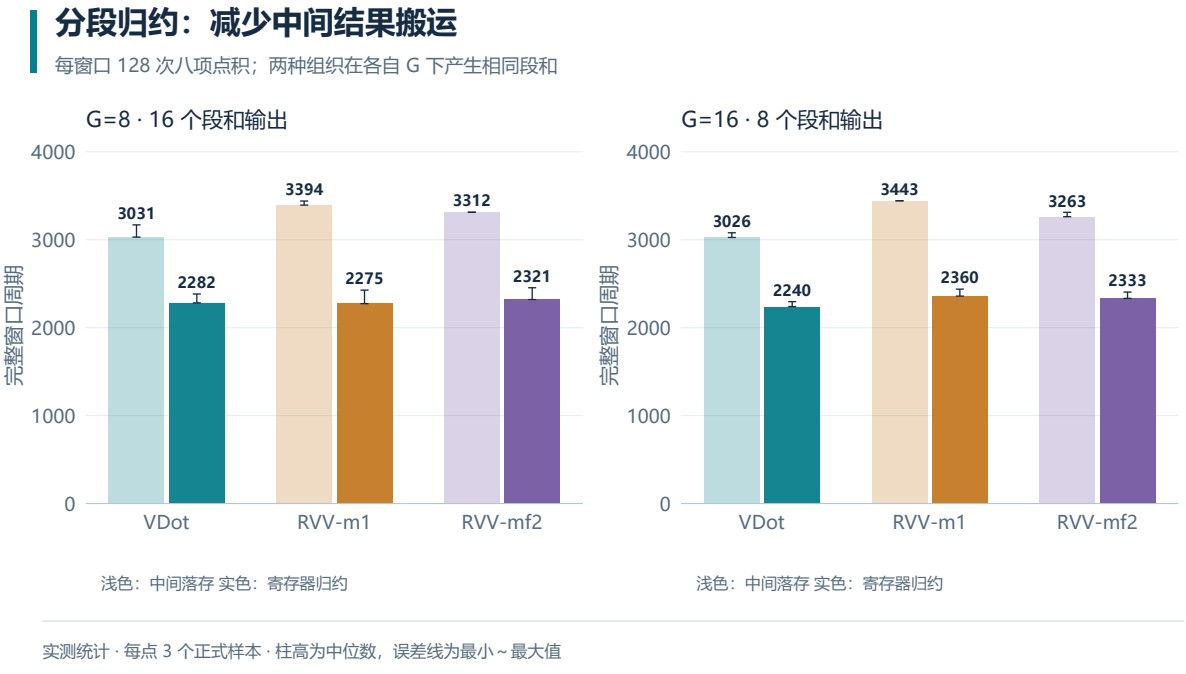


图 9-10　G=8、G=16 分面展示，各面只比较相同段和输出。柱高为中位数，误差线为三个正式样本范围。

部分变体的首个正式样本高于后续样本。例如 G=8 的 VDot 为 2383、2282、2282 周期，RVV-m1 为 2427、2273、2275。比较跨实现的小幅差异时，需要同时查看中位数与样本范围。原始样本见分段核验记录〔26〕。

分段任务最终只需要段和，所以中间点积可以直接在寄存器中相加，省去反复存取。9.5 的任务则要求保存每一次点积，两组任务的输出数量不同。分段程序减少中间搬运后，仍然需要为每次点积加载输入，接下来继续调整这些加载。

9.7 输入加载：尾部策略与寄存器轮转

加载实验固定 G=16：做 128 次点积，输出八个段和，三种方案都执行 256 条向量加载。tu 要求保留输入寄存器尾部旧值；ta 不要求保留未用的尾部；ta_rr 在 ta 基础上交替使用 v1/v2、v6/v7，两组寄存器轮流装数据，使下一组输入可以提前准备。VDot 与 RVV-mf2 使用相同改法；VDot 指令本身的 oldVd 规则不变。

本组重新测量 tu 对照，每变体预热五次、取九个正式样本。历史 G=16 的 2240/2333 周期来自不同程序上下文及较少预热，因此本节的优化比例统一由表 9-9 的同批数据计算。

表 9-9　同一 G=16 任务的加载对照，每种组织均为 256 条向量加载

组织	路径	周期中位数	最小～最大	退休数
tu	VDot	2239	2238～2241	843
tu	RVV-mf2	2262	2260～2262	997
ta	VDot	1235	1231～1236	843
ta	RVV-mf2	1396	1392～1398	997
ta_rr	VDot	1089	1087～1090	843
ta_rr	RVV-mf2	1355	1352～1357	997

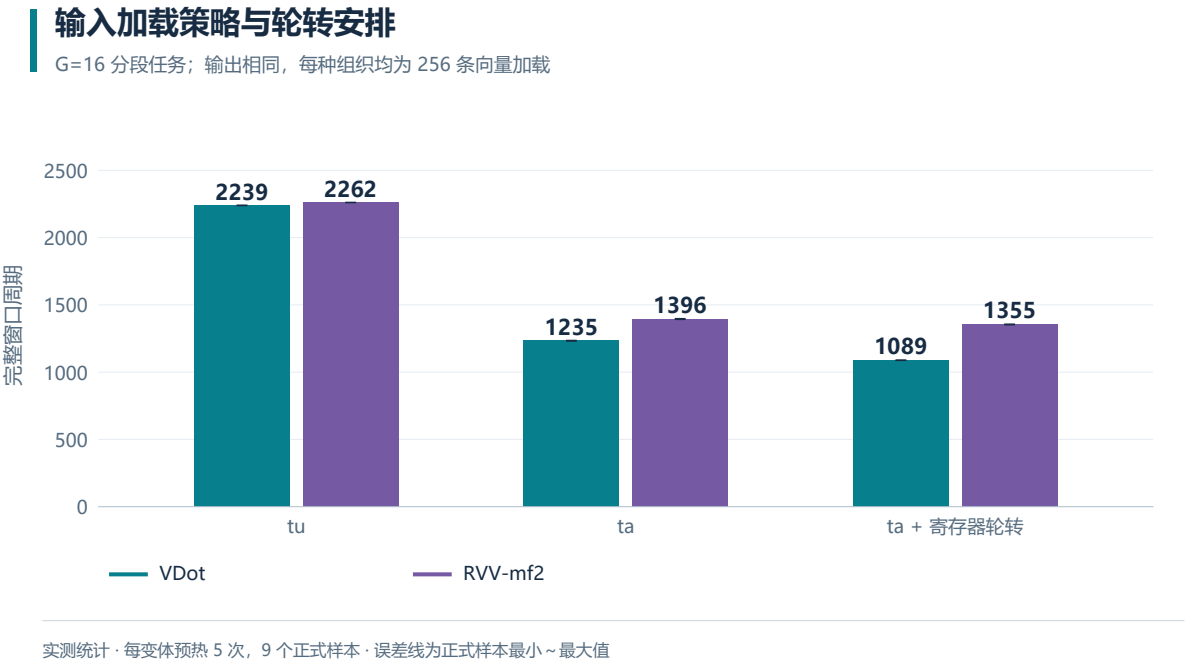


图 9-11　同输出、同加载条数下，策略和安排改变后周期下降。误差线为九个正式样本范围。

同一批测量中，VDot 从 tu 的 2239 周期降到 ta_rr 的 1089 周期，减少 51.4%；RVV-mf2 从 2262 降到 1355 周期。两条路径各自的退休数都没有变，变化来自输入策略与指令安排。ta 不再要求加载时保留未使用的尾部旧值；两组寄存器轮换，让程序在处理当前输入时提前准备下一组。实验确认组合优化缩短了时间，各种依赖和调度等待分别贡献多少，还需逐内核事件记录。

两条实现都使用 ta_rr 时，再横向比较：RVV-mf2 用 1355 周期，VDot 用 1089 周期，整窗比为 1355/1089=1.24，VDot 耗时少 19.6%。这与上一段“VDot 自己优化前后减少 51.4%”是两项不同的比较〔27〕。

9.8 2×2 小块：共享输入与加载复用

上一组每次点积仍各自加载输入，本组让四次点积共用数据。任务计算 K=256 的 2×2 小块，四个输出分别累加 A0/B0、A0/B1、A1/B0、A1/B1 的点积。每轮处理八个元素，运行 32 轮，共 128 次八项点积；最后保存四个 128 位结果。

若四次点积各自重新加载两份输入，一轮要加载八次。复用方案先把 A0、A1、B0、B1 四个块各加载一次，再组合出四次点积：A0 供前两次使用，A1 供后两次使用，B0 和 B1 也各被使用两次。因此一轮只加载四次，32 轮从 256 条向量加载减到 128 条。VDot 和 RVV-mf2 都采用 ta，B 输入提前按列存好，布局准备放在计时外。

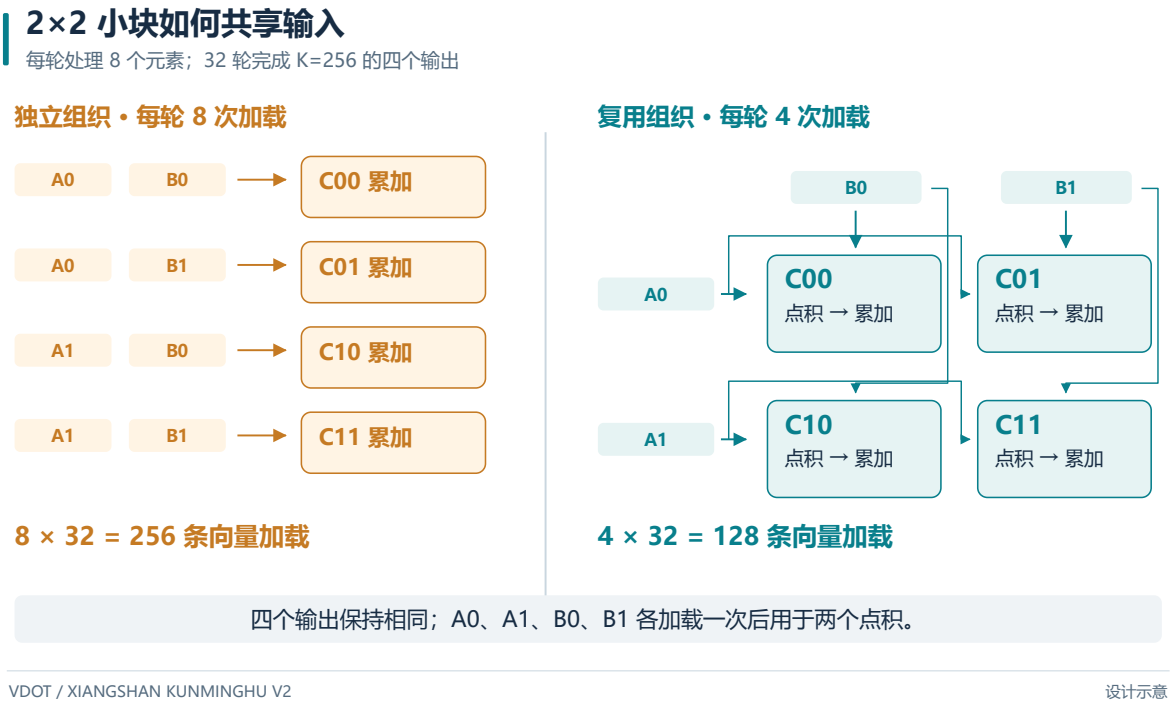


图 9-12　2×2 输入复用。每轮 A0、A1、B0、B1 各加载一次，供四个点积使用；32 轮的向量加载由 256 条减为 128 条。

表 9-10　同一四输出任务的输入复用比较，五次预热后取九个正式样本

组织	路径	周期中位数	最小～最大	退休数	向量加载数
独立加载	VDot	1077	1076～1087	950	256
独立加载	RVV-mf2	1268	1262～1270	1115	256
复用加载	VDot	790	787～792	694	128
复用加载	RVV-mf2	1043	1041～1044	859	128

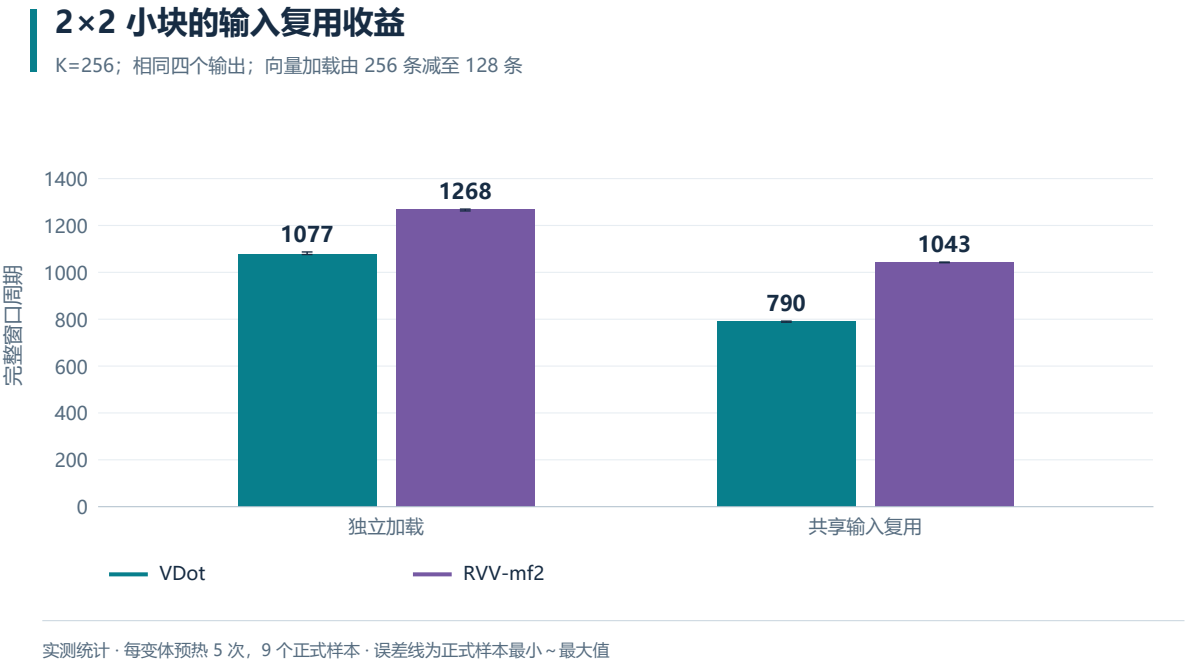


图 9-13　相同四个输出下的复用收益。柱高为周期中位数，误差线为九个正式样本范围。

共用输入后，VDot 从 1077 降到 790 周期，少 26.6%；RVV-mf2 从 1268 降到 1043 周期，少 17.7%。两条实现各少退休 256 条指令，减少的是向量加载及配套地址推进。两条都用复用方案时，RVV-mf2/VDot 为 1043/790=1.32，VDot 耗时少 24.3%。

本组在固定配置、固定种子和预热后的 2×2 内核中测量共享输入收益，输入按所需布局预先准备，矩阵转置或打包位于窗口外。完整样本、输出与保护区检查、输入和指定状态核验，以及差分结果均保存于配套记录〔28〕〔17〕。

9.9 性能结论与适用条件

完整操作依赖链中，VDot 为 7 周期，RVV-m1、RVV-mf2 分别为 55、47 周期，延迟比为 7.86、6.71。固定源、轮转目的时，VDot 平均每增加一次操作只增加 1 周期，体现了专用单元连续处理独立点积的能力。

加入访存后，初版逐点输出程序基本持平；双方采用相同优化后，G=16 加载轮转和 2×2 输入复用的整窗比分别达到 1.24、1.32。其中，预打包的 2×2、K=256 内核中，VDot 用 790 周期，RVV-mf2 用 1043 周期，VDot 耗时少 24.3%。计算单元的优势需要配合加载安排与输入复用，才能保留在完整内核中。

面向边缘 LLM 推理，VDot 可承担 INT8 线性层中反复执行的点积计算。现有结果说明，在处理器流水线中集成专用点积单元，并配合软件组织输入，能够缩短矩阵小块的计算时间，为量化线性算子提供加速基础。当前收益对应预热后的指令和小内核测量，完整模型的推理收益仍需模型级测试。

附录 C 模型与性能索引

本附录列出第三阶段的固定环境、应用与复现入口。第三阶段复用第二阶段的修复版 emu、源码 NEMU 和共同 Flash，性能运行固定 seed=1。各应用的版本、样本与退出状态分别记录在对应实验目录。

C.1 固定环境与模型

表 C-1　第三阶段主要身份

对象	版本或保存的 SHA256
XiangShan 基版本	`16ae9ddcda54fc9a2fddffad73174cf793ac7814`，叠加完整 RTL 补丁
NEMU 基版本	`a48a11f208ff81ad765591d83271d1b339fb3861`，叠加 VDot 参考语义
emu	`f9653532fd2a4b9f0d0cc08c131f7c258d4b62c46e672c6886d6643aad926850`
服务器参考库	`11b9f6807c36ed860e0c64cc4e58d27216b19335ecde1ce5797ba2d1cd057b74`
Flash	`fea548655a8b6eecacace83e2f4380a18b8aa83bce964c8aae4cffbe2f5a9724`
本地 API/等价检查参考库	`5ba6caf77513904c6ea541c4c145720a6bebe63f983eefa5b73e53441069c872`

本地 API 和服务器运行分别采用上述参考库身份。具体依赖和 NEMU 配置见源码交付记录〔5〕；每个应用的运行参数、二进制身份和退出状态位于各自的原始记录中。

C.2 应用、样本和复现入口

各应用均独立生成 RISC-V 镜像，由固定 emu 加载，并显式设置 `--diff` 与 `-F`。原运行入口保存了输入包、构建参数和日志目录的创建方式。在新环境独立复验时，应使用新的运行目录并核对参考库身份。

表 C-2　应用与对应证据

实验	源码	运行与数据
RVV 等价	vdot-rvv-equiv-v1〔14〕	命令〔15〕、结果〔13〕
计时预检	vdot-timing-pilot-v1〔29〕	运行入口〔30〕、统计〔31〕
依赖链	vdot-chain-v1〔32〕	命令〔33〕、统计〔20〕
八路批处理	vdot-throughput-v1〔34〕	命令〔35〕、统计〔23〕
小数组	vdot-e2e-v1〔36〕	命令〔37〕、统计〔24〕
分段点积	vdot-segment-v1〔38〕	命令〔39〕、完整样本〔26〕
输入加载与轮转	方法及源码入口〔40〕	全部轮次〔27〕
2×2 输入复用	方法及源码入口〔40〕	全部轮次〔28〕

两组加载/复用应用共 168 次调用，其中预热 60 次、正式 108 次，来源为汇总核验〔17〕。加载恢复运行复用了已构建的镜像，首次中断的目录和原构建耗时未完整归档；成功运行的源码、BIN、ELF、样本和身份均已保存。

C.3 应用镜像与完整原件

表 C-3　主要应用 BIN 的保存 SHA256

应用	SHA256
RVV 等价	bf29b9367a4b9045b139224bb5b2b6502bdeae005969abdc6129ba0fd6c76010
计时预检	8545abf1afb71ec405a8c81bada3fe5054a332964436eb7976a824edaf91bda6
依赖链	13088ad74214b3c7626d675e09a30a130089ec5feeff9ee8434a373c16ea1217
八路批处理	e9610f34acc9eb2118d1850f802eab90f30adc40074641f88d3f91e86d8abeab
小数组	8786fb22c9ca4fe32ae07d13078a8c619bedd2f5832897a6d494d02a268ec762
分段点积	bd034d3d3bebae84c68d26d5b2d52c6420f26ea89c41178621a217321c0aac8b
输入加载	6436b127636a046afa8e9451270be3a48023ff21f4b86a0c1c194e41d217f54f

模型及故障定位原件包括 NEMU 源码快照、API 结果、差分失败诊断、修复回归及流水套件〔3〕〔6〕〔9〕〔7〕〔8〕。

早期等价及四组计时的完整来源文件保存在本地原件归档中〔41〕，其完整逐轮样本支持复算〔16〕。后续分段和加载/复用各有独立的原始目录。复核已有结论时优先从这些原件读取；需要新实验时，再明确配置、输入、样本和计时边界。

材料完整路径

编号与本阶段正文一致。路径相对于配套 RISCV_vdot 工程根目录；图片随本文档目录提供，完整源码、日志和大型 VCD 由配套工程提供。

编号	材料	项目相对路径
1	项目 v0.1 指令规格	docs/vdot-spec-v0.1.md
2	vdot.h 源码	artifacts/vdot-integration-2026-10-24/delivery/source-snapshot/NEMU/src/isa/riscv64/instr/rvv/vdot.h
3	NEMU 修改快照	artifacts/vdot-integration-2026-10-24/delivery/source-snapshot/NEMU
4	接口核对记录	artifacts/scnet-abi-2026-10-24/abi-review.json
5	交付与复现记录	reports/stage3/delivery.md
6	原始 API 结果	artifacts/vdot-integration-2026-10-24/nemu-api-151012/result.json
7	修复回归	artifacts/vdot-mtval-fix-2026-10-24/server-xYGYlZ/review.json
8	流水回归	artifacts/vdot-pipeline-2026-10-24/server-JuGUz6/review.json
9	差分诊断记录	artifacts/vdot-matrix-2026-10-24/server-OMcrpf/diagnosis.md
10	修复说明	artifacts/vdot-mtval-fix-2026-10-24/README.md
11	本项目汇编	benchmarks/rvv-baseline/vdot-rvv-equiv-v1/cases.S
12	本地结果	artifacts/vdot-rvv-equivalence-2026-10-26/local-170237/result.json
13	服务器核验	artifacts/vdot-rvv-equivalence-2026-10-26/server-8nvimb/review.json

编号	材料	项目相对路径
14	应用源码	benchmarks/rvv-baseline/vdot-rvv-equiv-v1
15	实际构建运行入口	artifacts/vdot-rvv-equivalence-2026-10-26/server-paste-command.txt
16	归档逐轮表	artifacts/stage3-remote-archive-2026-10-27/server-MIwNSR/complete-samples.md
17	补测结果	artifacts/vdot-load-reuse-2026-10-30/server-review.json
18	计时程序	benchmarks/rvv-baseline/vdot-timing-pilot-v1/cases.S
19	预检记录	artifacts/vdot-timing-pilot-2026-10-26/server-5vHUp8/review.json
20	原始统计	artifacts/vdot-chain-2026-10-26/server-KiI3xX/raw-statistics.json
21	依赖链程序	benchmarks/rvv-baseline/vdot-chain-v1/cases.S
22	运行记录	artifacts/vdot-chain-2026-10-26/server-KiI3xX/review.json
23	原始统计	artifacts/vdot-throughput-2026-10-26/server-uLgHm6/raw-statistics.json
24	原始统计	artifacts/vdot-e2e-2026-10-27/server-bijzGZ/raw-statistics.json
25	完整计数器路径	artifacts/stage3-closeout-2026-10-27/e2e-read/perf-context.json
26	分段核验	artifacts/vdot-segment-2026-10-29/server-Pw1wQ8/review.json
27	加载实验记录	artifacts/vdot-load-reuse-2026-10-30/load/server-L4TEPC/review.md
28	复用实验记录	artifacts/vdot-load-reuse-2026-10-30/tile/server-x9yIzN/review.md
29	vdot-timing-pilot-v1	benchmarks/rvv-baseline/vdot-timing-pilot-v1
30	运行入口	artifacts/vdot-timing-pilot-2026-10-26/retry-budget/server-paste-command.txt
31	统计	artifacts/vdot-timing-pilot-2026-10-26/server-5vHUp8/raw-statistics.json
32	vdot-chain-v1	benchmarks/rvv-baseline/vdot-chain-v1
33	命令	artifacts/vdot-chain-2026-10-26/server-paste-command.txt
34	vdot-throughput-v1	benchmarks/rvv-baseline/vdot-throughput-v1
35	命令	artifacts/vdot-throughput-2026-10-26/server-paste-command.txt
36	vdot-e2e-v1	benchmarks/rvv-baseline/vdot-e2e-v1
37	命令	artifacts/vdot-e2e-2026-10-27/server-paste-command.txt
38	vdot-segment-v1	benchmarks/rvv-baseline/vdot-segment-v1
39	命令	artifacts/vdot-segment-2026-10-29/server-paste-command.txt
40	方法及源码入口	reports/stage3/load-reuse-plan.md
41	本地原件归档	artifacts/stage3-remote-archive-2026-10-27/server-MIwNSR